

I-Nodes

1 Basic Idea

I-Node

An **i-node**, or **index-node**, is a per-file data structure that stores a file's attributes and the disk addresses of the blocks belonging to that file.

- Each file has its own i-node.
- The i-node tells the operating system where the file's blocks are located on disk.
- Given the i-node, the file system can find all blocks of the file.
- The file name is usually stored in a directory entry, while the file's metadata and block addresses are stored in the i-node.

2 I-Node Structure

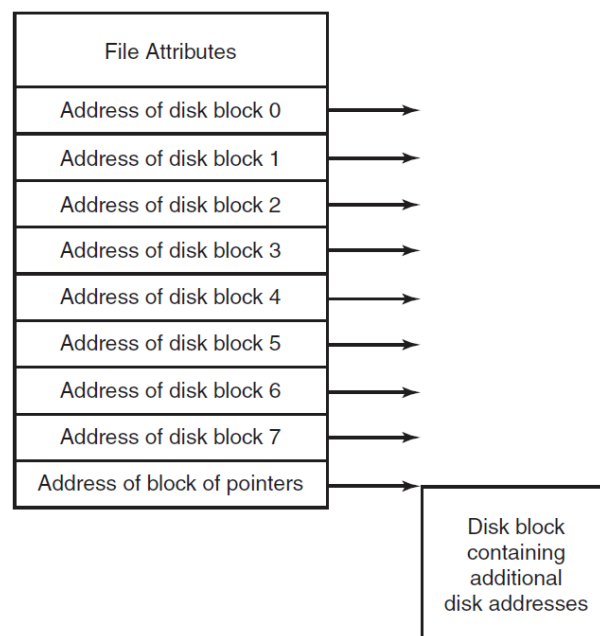


Figure 4-13. An example i-node.

Figure 1: An example i-node with direct block addresses and one pointer block.

- The top part stores **file attributes**.
- Attributes may include file size, owner, permissions, and time information.
- The next fields store direct addresses of disk blocks.

- Each direct address points to one data block of the file.
- The final address may point to a block that contains more disk-block addresses.

3 How Blocks Are Found

Lookup Rule

To access a file, the operating system first finds the file's i-node. The i-node then gives the disk block addresses needed to read or write the file data.

1. A directory entry maps the file name to its i-node.
2. The file system loads the i-node.
3. The i-node's direct block addresses point to the first data blocks.
4. If the file is larger, an indirect pointer points to a block containing additional addresses.
5. The operating system follows these addresses to reach the required file data.

4 Main Advantage

Memory Efficiency

An i-node only needs to be in memory while its file is open. This makes i-nodes more scalable than a full in-memory file allocation table.

- A FAT-style table has one entry for every disk block.
- Therefore, FAT memory use grows with disk size.
- With i-nodes, memory use grows mainly with the number of open files.
- If each i-node has size n bytes and at most k files can be open, the needed memory is:

Open I-Node Memory

$$\text{memory for open i-nodes} = k \times n \text{ bytes}$$

- This amount is usually much smaller than storing a full table for every disk block.
- The disk may be 100 GB, 1000 GB, or 10,000 GB, but the open i-node table depends on open files, not total disk blocks.

5 Problem: Very Large Files

Fixed Address Limit

If an i-node has room for only a fixed number of disk addresses, then a file can grow beyond the direct addresses stored in the i-node.

- A small file can be represented using only direct block addresses.
- A larger file needs more block addresses than the i-node can store directly.
- One solution is to use the last i-node address as an **indirect pointer**.
- The indirect pointer points to a disk block full of additional disk-block addresses.
- More advanced systems may use multiple levels of indirection.

6 Direct and Indirect Addresses

Address Type	Meaning
Direct address	Points directly to a file data block.
Indirect address	Points to a block that contains more disk-block addresses.
Double indirect address	Points to a block whose entries point to more address blocks.
Triple indirect address	Adds another level for very large files.

7 Comparison with FAT

Point	FAT	I-Nodes
Main structure	One table entry per disk block	One structure per file
Memory use	Proportional to disk size	Proportional to open files
When loaded	Normally kept in memory	Loaded when file is open
Scaling	Poor for very large disks	Better for large disks
File growth	Follow table chain	Use direct and indirect addresses

8 Key Points

- An i-node stores file attributes and block addresses.
- It separates file naming from file storage information.
- It only has to be in memory when the file is open.
- It scales better than keeping one memory-table entry for every disk block.
- Indirect blocks allow the scheme to support large files.

Final Point

I-nodes solve the scaling problem of a full in-memory allocation table by keeping per-file block information in an i-node and loading only the i-nodes for currently open files.